

1. Tutorial básico: tablas, fórmulas y resaltado

Este capítulo presenta los elementos fundamentales del DSL mediante un ejemplo mínimo. Al terminar sabrá definir una tabla con columnas, añadir una columna calculada, condicionar el resaltado de celdas y generar el archivo de salida.

1.1 Qué vamos a construir

Queremos una tabla con cuatro columnas:

Nombre	Nota 1	Nota 2	Promedio
Ana	85	90	87.5
Luis	70	75	72.5
Elena	92	88	90.0
Carlos	60	65	62.5
Sofía	78	82	80.0
Pedro	95	91	93.0

Además:

- La columna **Promedio** no se escribe a mano: se calcula automáticamente como el promedio de las dos notas.
- Hay un **número fijo** (80, por ejemplo) almacenado en el libro.
- Los nombres de las personas cuyo promedio supera ese número aparecen **resaltados en rojo**.

1.2 Estructura del programa

El programa completo consta de tres partes, que escribiremos en un solo archivo `tutorial-basico.lisp`:

1. Los **datos** de entrada (nombres y notas).
2. La **definición de la tabla** con su columna calculada y su regla de resaltado.
3. El **libro** que reúne todo y las instrucciones para generar la salida.

1.3 Los datos

Los datos se guardan en una variable de Lisp con `defparameter`. Cada fila es una lista con cuatro celdas (una por columna). Las primeras tres filas son de prefijo: dos filas en

blanco y una fila de encabezados. Las filas siguientes son los datos propiamente dichos.

Listing 1.1: tutorial-basico.lisp — datos de entrada.

```
1 (defparameter *DATOS*
2   '((" " " " " " ")
3     (" " " " " " ")
4     ("Nombre" "Nota 1" "Nota 2" "Promedio")
5     ("Ana" "85" "90" " ")
6     ("Luis" "70" "75" " ")
7     ("Elena" "92" "88" " ")
8     ("Carlos" "60" "65" " ")
9     ("Sofia" "78" "82" " ")
10    ("Pedro" "95" "91" " ")))
```

Observe que la columna Promedio se deja vacía () en los datos, porque su valor lo calculará el DSL automáticamente.

Formato de las celdas vacías

Las celdas que corresponden a columnas calculadas se marcan con (cadena vacía). El generador las sobrescribe con la fórmula correspondiente.

1.4 Definir la tabla

La macro `def-table` declara una plantilla de tabla. Le damos un nombre y la lista de columnas. Cada columna se escribe como un par (`nombre .Etiqueta`), donde `nombre` es el identificador interno (un símbolo del DSL) y `.Etiqueta` es el texto que se mostrará como encabezado.

Listing 1.2: Definición de la tabla.

```
1 (def-table tabla-notas
2   ((nombre "") (nota1 "") (nota2 "") (promedio ""))
3   :cell-height 1
4   :cell-width 1)
```

Por ahora la tabla solo declara las columnas, sin fórmulas ni resaltados. A continuación añadiremos esas características.

1.5 Columna calculada

Queremos que `promedio` se calcule solo:

$$\text{promedio} = \frac{\text{nota1} + \text{nota2}}{2}$$

En lenguaje natural diríamos: “para cada fila, suma `nota1` y `nota2`, y divide el resultado entre 2”. En el DSL esto se expresa con la opción `:computed` y las formas `add` y `divide`.

Pero, ¿cómo hacemos referencia al número 2? El DSL permite definir *parámetros* que se almacenan como celdas fijas en el libro. Usaremos un parámetro llamado `divisor` con valor 2.

También necesitaremos un parámetro `nota-minima` para el resaltado, así que lo añadimos ya.

Listing 1.3: Tabla con columna calculada.

```
1 (def-table tabla-notas
2   ((nombre "") (nota1 "") (nota2 "") (promedio ""))
3   :cell-height 1
4   :cell-width 1
5   :computed
6     ((promedio (divide (add (col nota1) (col nota2))
7                       (param divisor)))))
```

Cada pieza significa:

- `(col nota1)` — toma el valor de la columna `nota1` en la fila actual.
- `(add (col nota1) (col nota2))` — suma ambos valores.
- `(param divisor)` — referencia al parámetro `divisor` (que definiremos más adelante).
- `(divide ... (param divisor))` — divide la suma entre el divisor.

¿Por qué no escribir simplemente 2?

El DSL no tiene literales numéricos directos en las expresiones; los valores fijos se definen como parámetros y se referencian con `param`. Esto mantiene el AST libre de constantes incrustadas y facilita cambiar los valores sin modificar las fórmulas.

1.6 Resaltado condicional

Ahora queremos que los nombres de las personas cuyo promedio supere el mínimo aparezcan en rojo. En lenguaje natural: “si el promedio de esta fila es mayor que `nota-minima`, colorea el nombre”.

La opción `:render` de `def-table` acepta una forma `conditional-rendering` que especifica la condición y las columnas a colorear.

Listing 1.4: Tabla con columna calculada y resaltado.

```
1 (def-table tabla-notas
2   ((nombre "") (nota1 "") (nota2 "") (promedio ""))
3   :cell-height 1
4   :cell-width 1
5   :computed
6     ((promedio (divide (add (col nota1) (col nota2))
7                       (param divisor)))))
8   :render
9     (conditional-rendering
10      :condition (gt (col promedio) (param nota-minima)))
```

```
11 :target-columns (nombre)))
```

Al leer la condición en voz alta:

- `gt` significa “mayor que” (*greater than*).
- `(col promedio)` es el promedio de la fila actual.
- `(param nota-minima)` es el valor fijo que definiremos.
- `:target-columns (nombre)` indica qué columna se colorea cuando la condición se cumple.

1.7 El parámetro y la generación

Con la tabla definida, solo falta ensamblar el libro, proporcionar los parámetros y lanzar la generación.

Los parámetros se pasan al instanciar la tabla con `tabla` mediante la clave `:params`. Cada parámetro es un par (`nombre valor`).

Listing 1.5: Libro completo con parámetros.

```
1 (libro tutorial-basico
2   :filename "Tutorial_Basico.xlsx"
3   :hojas (list
4     (hoja "Notas"
5       (tabla tabla-notas
6         :data *DATOS*
7         :params ((nota-minima 80) (divisor 2)))))
8
9 (xl-generate tutorial-basico "tutorial-basico.py")
10 (xl-run-generated "tutorial-basico.py")
```

¿Dónde quedan los parámetros en el libro?

Los parámetros se colocan como celdas auxiliares en la misma hoja, a continuación de la tabla. La referencia (`param nombre`) se traduce a una referencia absoluta a esa celda (ej. `$E$2`).

1.8 El programa completo

A continuación se muestra el contenido íntegro del archivo `tutorial-basico.lisp`:

Listing 1.6: `tutorial-basico.lisp` — programa completo.

```
1 (load (merge-pathnames "dsl-directo.lisp" *load-truename*))
2
3 (defparameter *DATOS*
4   '((" " " " " ")
5     (" " " " " ")
6     ("Nombre" "Nota 1" "Nota 2" "Promedio"))
```

```

7      ("Ana"      "85" "90" "")
8      ("Luis"     "70" "75" "")
9      ("Elena"    "92" "88" "")
10     ("Carlos"   "60" "65" "")
11     ("Sofia"    "78" "82" "")
12     ("Pedro"    "95" "91" "")))
13
14 (def-table tabla-notas
15   ((nombre "") (nota1 "") (nota2 "") (promedio ""))
16   :cell-height 1
17   :cell-width 1
18   :computed
19     ((promedio (divide (add (col nota1) (col nota2))
20                          (param divisor))))
21   :render
22     (conditional-rendering
23      :condition (gt (col promedio) (param nota-minima))
24      :target-columns (nombre)))
25
26 (libro tutorial-basico
27   :filename "Tutorial_Basico.xlsx"
28   :hojas (list
29     (hoja "Notas"
30       (tabla tabla-notas
31         :data *DATOS*
32         :params ((nota-minima 80) (divisor 2)))))
33
34 (xl-generate tutorial-basico "tutorial-basico.py")
35 (xl-run-generated "tutorial-basico.py")

```

1.9 Ejecución

Para ejecutar el tutorial:

```

cd tutorial/
sbcl --script tutorial-basico.lisp

```

Esto cargará el DSL, construirá el AST, generará `tutorial-basico.py` y lo ejecutará para crear `Tutorial_Basico.xlsx`.

1.10 Resultado esperado

El archivo `Tutorial_Basico.xlsx` contiene una hoja llamada “Notas” con la siguiente estructura:

Rango	Contenido
Filas 4–9, columnas A–D	Tabla con 6 personas
Columna D, filas 4–9	Promedio calculado con fórmula
Columna A, filas 4–9	Nombres en rojo si promedio > 80

Con los datos de ejemplo, los nombres de Ana, Elena, Sofía y Pedro aparecen resaltados en rojo porque su promedio supera 80. Luis y Carlos quedan con formato normal.

1.11 Resumen de formas utilizadas

Forma	Para qué sirve
<code>(def-table nombre (cols...) opts...)</code>	Define una plantilla de tabla
<code>:computed ((col expr))</code>	Columna calculada con una expresión
<code>(conditional-rendering :condition :target-columns)</code>	Regla de resaltado condicional
<code>(add a b)</code>	Suma dos expresiones
<code>(divide a b)</code>	Divide dos expresiones
<code>(gt a b)</code>	Verdadero si a es mayor que b
<code>(col nombre)</code>	Valor de la columna en la fila actual
<code>(param nombre)</code>	Valor de un parámetro fijo
<code>:params ((nombre valor)...) (libro nombre ...)</code>	Define parámetros al instanciar la tabla
<code>(hoja nombre tablas ...)</code>	Define el libro completo
<code>(hoja nombre tablas ...)</code>	Define una hoja
<code>(tabla plantilla :data :params)</code>	Instancia una tabla con datos y parámetros
<code>(xl-generate wb archivo)</code>	Genera el script Python
<code>(xl-run-generated archivo)</code>	Ejecuta el script y produce el .xlsx